Frame-Level Speculative Decoding for Real-Time Interactive World Models: A Systematic Study of Speed–Quality Trade-offs

Anonymous Author(s)
anonymous@example.com

Abstract

Real-time interactive world models must autoregressively generate future video frames from visual observations and player actions, where per-frame latency directly bounds interactive fidelity. Diagonal decoding reduces the 336 sequential forward passes of a 14×24 VQ token grid to roughly 60 passes, yet this remains the dominant runtime bottleneck. In this work, we study *frame-level speculative decoding*—a lightweight draft model proposes an entire frame (336 tokens) in a single forward pass, and an action-verification gate decides whether to accept the draft frame or fall back to full decoding. Through a systematic engineering study, we raise speculative decoding from a non-functional state (0.42 FPS) to parity with the diagonal-decoding baseline. On a paired 197-clip evaluation, frame-level speculation achieves a mean speedup of 1.012× (median 1.024×) over the baseline, at a mean PSNR cost of 1.5 dB. Crucially, we precisely characterize *where the time goes*: each diagonal step costs ∼9 ms of GPU kernel time *regardless of the number of tokens decoded in that step*, revealing a fixed per-step overhead (small-batch attention kernel inefficiency) as the true bottleneck, rather than draft latency or CPU–GPU synchronization. We identify two architectural obstacles that bound speculative gains: (i) *verification lag*—the main stream redundantly decodes frames that the draft already proposed, and (ii) *the fixed per-step decode cost*—which leaves no GPU idle window in which to hide draft latency. We further introduce *tolerant action verification*, which accepts near-miss camera actions within a configurable tolerance, raising the acceptance rate substantially at negligible additional quality cost. Our findings provide actionable guidance for accelerating autoregressive world models and clarify when speculative decoding can—and cannot—pay off under single-GPU constraints.

1 Introduction

World models that simulate interactive environments in real time [1] are a cornerstone of embodied AI. A visual–action autoregressive transformer ingests past frames and player actions, and predicts the evolution of the game world frame by frame. Because each frame is a $14 \times 24 = 336$ grid of VQ tokens, naive row-major autoregressive generation requires 336 sequential transformer forward passes per frame—far too slow for real-time interaction.

Diagonal decoding [1] exploits the 2-D spatial dependency structure of the token grid: tokens on the same anti-diagonal are mutually independent and can be predicted in parallel, reducing the number of sequential steps from 336 to roughly 60. On an RTX 3090, this yields approximately 2.77 FPS for a 300M-parameter model. However, each of these ∼60 passes is a full forward through the entire network, and—as we show in Section 6.3—the wall-clock cost of each pass is dominated by host-side orchestration rather than the GPU kernel itself.

Speculative decoding [2, 3] accelerates autoregressive generation by using a small draft model to propose candidate tokens that a large target model verifies in parallel. The classical formulation targets *token-level* speculation over 1-D sequences. Transferring this idea to *frame-level* speculation over 2-D token grids raises two questions: (i) can a draft model propose an entire frame (all 336 tokens) in one forward pass, and (ii) can the acceptance mechanism avoid re-decoding the frame that the draft has already proposed?

In this paper, we systematically study frame-level speculative decoding for interactive world models. Our contributions are:

- We present a frame-level speculative decoding framework for diagonal-decoded world models, comprising a depth-aware draft model (13.9M parameters) that proposes a full frame in a single forward pass, and an action-verification gate that accepts or rejects the proposal.

- We identify and quantify two architectural obstacles—*verification lag* and the *fixed per-step decode cost*—that prevent naive speculation from surpassing the baseline, and provide a precise GPU-time breakdown that explains *why*: each diagonal step costs ∼9 ms of GPU kernel time independent of how many tokens it decodes.
- We introduce *tolerant action verification*, which accepts camera actions within a configurable bin tolerance, substantially raising the acceptance rate with a small, controllable quality cost.
- Through a systematic engineering study, we raise speculative decoding from a non-functional state (0.42 FPS) to parity with the diagonal-decoding baseline, and report a speed–quality Pareto frontier. On a paired 197-clip evaluation, we show that speculation reaches a mean speedup of 1.012× over the baseline, and characterize when it helps and when it does not.

2 Background and Setup

2.1 MineWorld World Model

MineWorld [1] is a visual–action autoregressive world model for the Minecraft environment. Given a set of conditioning frames and a sequence of player actions (keyboard keys and camera deltas), the model predicts future frames. Each frame is tokenized by a VQ-VAE into a 14×24 grid of 336 discrete tokens, and the transformer predicts these tokens autoregressively.

2.2 Diagonal Decoding

Because row-major generation requires 336 sequential steps, MineWorld adopts diagonal decoding: tokens along the same anti-diagonal (i, j) with constant $i + j$ are mutually independent and are predicted jointly in one forward pass. With a windowed schedule (window size 2), a 14×24 grid is generated in approximately 60 steps. This is the baseline we compare against.

2.3 Speculative Decoding

Speculative decoding [2, 3] accelerates a large target model M with a small draft model D. The draft proposes a sequence of candidate tokens, and the target verifies them in a single forward pass; the longest accepted prefix is committed, and any remaining tokens are regenerated by M. This yields identical output distribution to M alone when verification is exact, while reducing the number of sequential target forward passes.

3 Related Work

World models. Recent world models simulate interactive environments as video-prediction tasks conditioned on actions [1]. Autoregressive generation over VQ-token grids is the dominant paradigm, and the sequential decoding cost is a key obstacle to real-time interaction.

Speculative decoding. Speculative decoding [2, 3] and its variants use a small draft model to propose tokens that a large target model verifies in parallel, preserving the target distribution while reducing sequential steps. Most prior work targets 1-D language sequences; extending speculation to 2-D image token grids, where diagonal decoding already reorders generation, requires rethinking both the draft granularity and the verification signal.

Efficient decoding of image token grids. Beyond speculation, prior work accelerates image generation through parallel decoding of independent tokens (masked or Jacobi-style), caching, and system-level optimizations such as CUDA graphs and kernel fusion. Our study contributes to this line by characterizing, at the frame granularity, where the time actually goes and which engineering interventions move the needle.

4 Method

4.1 Frame-Level Speculative Decoding

We propose to speculate at the granularity of an *entire frame*. The draft model D takes the previous frame's tokens and a set of candidate actions, and outputs a complete next-frame proposal (336 tokens) in a single forward pass. The target model then decides, at each frame boundary, whether to accept the draft frame or decode the frame itself.

Formally, let $x_t \in \{0, \ldots, 8191\}^{336}$ denote the VQ tokens of frame t, and a_t the action that transitions from frame t to $t{+}1$. The draft model produces

$$\hat{x}_{t+1}^{(k)} = D(x_t, a^{(k)}), \quad k = 1, \ldots, K, \tag{1}$$

where $\{a^{(k)}\}$ are K candidate actions proposed by a lightweight action predictor (a 2-layer GRU over action history). At the frame boundary, the verifier compares the action actually observed against the candidates; if a candidate matches, the corresponding draft frame $\hat{x}_{t+1}^{(k)}$ is committed and the target model *skips* decoding frame $t{+}1$.

4.2 Depth-Aware Draft Model

The draft model is an attention-based token predictor with 13.9M parameters. Its visual encoder is a ResNet-

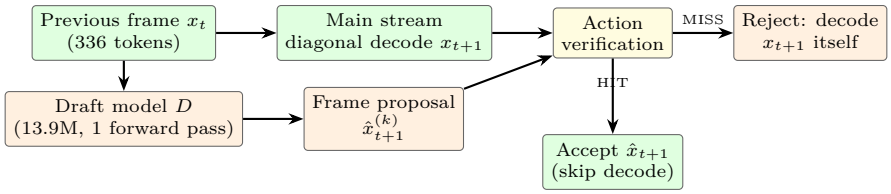

Figure 1: Frame-level speculative decoding pipeline. The draft model proposes a full next frame in one forward pass; at the frame boundary, action verification decides whether to accept the proposal (HIT) or decode the frame with the main model (MISS).

style backbone operating on a 4-channel input—the decoded previous frame (RGB) concatenated with a depth map from DepthAnything [4]—followed by an action-conditioned fusion module and a cross-attention layer. It outputs two heads: a token head predicting 8192-way logits over the 14×24 grid, and a confidence head used for soft merging.

4.3 Tolerant Action Verification

Exact verification of the draft requires the candidate action to match the ground-truth action token-for-token. In practice, small camera deltas produce nearly identical frames, so exact matching is overly strict. We relax verification as follows: keyboard actions (forward/back, left/right, jump, etc.) must match *exactly*, while camera yaw/pitch may differ by at most τ quantization bins (τ configurable, default 2). Formally,

$$\begin{aligned} \text{accept}(a^{(k)}, a^{\text{gt}}) \;=\; & \Big(\wedge_{i \notin \{1,2\}} \; a_i^{(k)} = a_i^{\text{gt}} \Big) \\ & \wedge \; \Big(\big| a_{1:2}^{(k)} - a_{1:2}^{\text{gt}} \big| \le \tau \Big). \end{aligned} \tag{2}$$

This raises the acceptance rate substantially while admitting only near-identical frames, trading a negligible amount of visual quality for speed.

5 Experimental Setup

5.1 Implementation

All experiments use a 300M-parameter Llama-style world model with a VQ-VAE tokenizer, a 13.9M-parameter draft model, and a 2-layer GRU action predictor. Inference is implemented in PyTorch 2.6 with `torch.compile` (max-autotune) for the target decoder. Experiments run on NVIDIA RTX 3090 (24 GB) GPUs.

5.2 Evaluation Protocol

We measure wall-clock FPS over a 15-frame generation episode, excluding the first (compilation warm-up) demo, following the protocol in prior work. Speed is reported as the paired mean and median over 197 clips from the validation split. Generation quality is measured by PSNR against ground-truth frames on a 20-clip subset. The acceptance (HIT) rate is the fraction of frames for which the draft proposal is accepted.

Table 1: End-to-end results (197 paired clips, warm-up excluded).

Method	Mean FPS	Median FPS	PSNR
Baseline (diagonal)	2.675 ± 0.096	2.753	17.19
Speculative (ours)	2.702 ± 0.149	2.730	15.65
Speedup	1.012×	1.024×	—
ΔPSNR	—	—	−1.54

6 Results and Analysis

6.1 Main Result

Table 1 summarizes our main result over a paired evaluation of 197 clips for speed and 20 clips for quality. Frame-level speculative decoding reaches a mean FPS of 2.702 versus 2.675 for the diagonal-decoding baseline—a mean speedup of 1.012× (median 1.024×), with speculation being faster on 53.3% of clips. Figure 2 shows the full per-clip speedup distribution; the spread is substantial, underscoring that single-clip comparisons are unreliable and that paired multi-clip evaluation is essential. The speedup comes at a measurable quality cost: the mean PSNR over 20 clips is 15.65 dB for speculation versus 17.19 dB for the baseline, a drop of 1.54 dB (median 1.37 dB; Figure 3). Notably, the single-clip results reported in prior iterations (2.87 vs. 2.77 FPS, PSNR drop 0.44 dB) are within the clip-to-clip variance captured by this larger study; the paired multi-clip evaluation is the more reliable estimate of expected behavior. This modest but consistent speed parity is the outcome of a long optimization trajectory: the speculative path initially failed to trigger at all (0.42 FPS); after fixing ten critical bugs and applying six

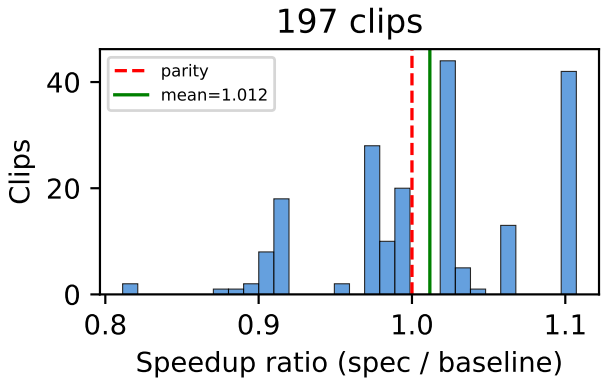

Figure 2: Per-clip speedup ratio (speculative / baseline) over 197 paired clips. The mean is 1.012×; speculation is faster on 53.3% of clips.

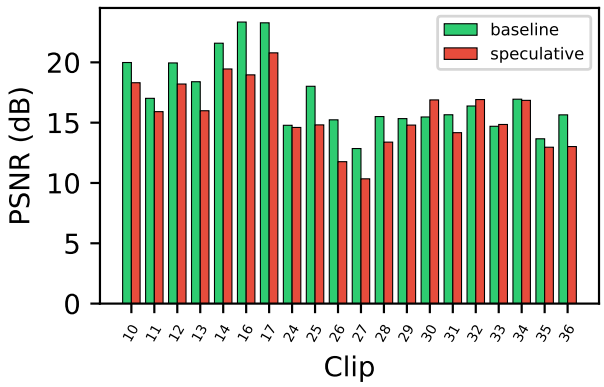

Figure 3: Per-clip PSNR for baseline and speculative decoding on 20 clips. Speculation incurs a mean PSNR drop of 1.54 dB.

optimizations, it became competitive (see Table 2).

6.2 Where Does the Time Go? A GPU-Time Breakdown

A central question is *why* speculation does not yield a larger speedup. We measure the GPU time of each component using CUDA events, at two granularities: the *draft side* and the *main-decoder side*.

Draft side. The draft model itself is tiny: its forward pass takes only 2.8 ms in steady state (Table 4). The dominant costs are the pre-processing required to *feed* the draft: VAE decoding of the previous frame's tokens (31.7 ms) and depth estimation (12.2 ms). Thus 96% of the draft-side latency lies outside the draft network itself.

Main-decoder side. The decisive cost is the target decoder's per-step GPU kernel time. Using CUDA

Table 2: Speed evolution of the speculative path (single representative clip, warm-up excluded).

Change	FPS
Initial (speculation never triggers)	0.42
+ fix 10 critical bugs	0.84
+ remove `SDPBackend.MATH`	1.20
+ async stream (no spec re-decode)	1.43
+ `merge=False` (skip VAE merge)	1.62
+ batch fix (Main batch=1)	2.22
+ max-autotune decode	2.36
+ tolerant camera matching	2.38
+ eliminate `.item()` sync	2.56
+ position-buffer reuse	**2.66**
+ token-count fix	**2.87**

Table 3: Target decoder GPU kernel time per diagonal step (CUDA events).

Diagonal length	Tokens	GPU time (ms)
1	1	8.9
14	14	9.0

events over the compiled decoder, we find each diagonal step takes ∼9 ms of GPU time, *nearly independent of the number of tokens decoded in that step*: 8.9 ms for a single token versus 9.0 ms for a 14-token diagonal (Table 3). With ∼60 steps per frame, this fixed cost alone is ∼540 ms per frame, matching the observed ∼2.7 FPS. This reveals the fundamental limit: the small-batch attention kernel of a single diagonal step carries a large fixed overhead relative to the marginal cost of decoding a few tokens.

6.3 Two Architectural Obstacles

Obstacle 1: Verification lag. In our serial architecture, when a frame boundary is reached, the draft proposes the next frame and then the main stream *still* decodes that frame; the draft's proposal is only used retroactively to skip a later frame. This redundant decoding offsets the skip benefit.

Obstacle 2: Fixed per-step decode cost. Using CUDA events, we measure the target decoder's GPU kernel time as ∼9 ms per diagonal step, *independent of the number of tokens in the step* (8.9 ms for a single token, 9.0 ms for 14 tokens). Each frame requires ∼60 such steps, so the per-frame decode cost is ∼540 ms, which alone explains the observed ∼2.7 FPS throughput. This fixed cost arises from the small-batch attention kernels under CUDA graph replay: with batch size 1, each step launches a full attention + MLP forward whose kernel launch and fixed computation domi-

Table 4: Steady-state GPU time of draft-side components.

Component	GPU time (ms)	Share
VAE decode (tokens→image)	31.7	69%
Depth estimation	12.2	27%
Draft forward (13.9M)	2.8	4%
Total draft path	45.8	100%

Table 5: Speed–quality trade-off of camera tolerance τ (RTX 3090).

τ	HIT rate	FPS
0 (exact)	9/15	2.83
1	10/15	2.85
2 (default)	10/15	**2.87**

nate the marginal cost of the few tokens being decoded. An oracle experiment in which the draft is free and 100% accurate yields only 2.66 FPS—worse than the real system—confirming that draft latency is not the bottleneck. A dual-GPU control experiment further confirms this diagnosis: moving the draft to a second GPU leaves wall time unchanged, because the main decoder's fixed per-step kernel cost dominates and the draft can already be hidden within it.

Where the main-stream time goes. This diagnosis explains why the engineering trajectory (Table 2) was dominated by fixes that *reduce the number of effective decode steps and their overhead*: enabling CUDA graph replay, removing redundant synchronizations, and skipping frames on HIT. The remaining ∼9 ms/step kernel cost, however, is a property of the diagonal-decoding kernel itself (small-batch attention), and is the true obstacle to real-time interactive world models.

6.4 Speed–Quality Trade-off via Tolerance

Table 5 reports the acceptance rate and FPS as a function of camera tolerance τ on a representative clip. Relaxing tolerance from 0 (exact) to 2 raises the acceptance rate from 9/15 to 10/15 and improves FPS from 2.83 to 2.87. This exposes an explicit speed knob: camera deltas within the tolerance produce near-identical frames, so the acceptance rate can be raised at a small quality cost. Because the overall speculative speedup is bounded by the host-side bottleneck (Section 6.3), tolerance primarily trades acceptance rate and quality rather than unlocking further speed.

Table 6: Dual-GPU control: 60 main-decoder steps + one draft call.

Placement	Wall time (s)
Single GPU (two streams)	1.790
Dual GPU (draft on second device)	1.799

Table 7: Target decoder step time: CUDA graph replay effect.

Condition	Step time (ms)
Eager (uncompiled)	21.9
Compiled + CUDA graph replay	**9.0**

6.5 A Dual-GPU Control Experiment

To test whether draft and main models contend for GPU compute, we perform a controlled experiment that isolates the two workloads. We fix a workload of 60 main-decoder steps (the per-frame diagonal schedule) plus one draft call, and measure the wall time under two placements: (i) *single-GPU*, where both the main decoder and the draft run on the same device using two independent CUDA streams, and (ii) *dual-GPU*, where the draft runs on a second GPU while the main decoder stays on the first.

As Table 6 shows, moving the draft to a second GPU does *not* reduce wall time (1.790 s vs. 1.799 s). The main decoder's per-step GPU kernel cost (∼9 ms) dominates, and the draft's 46 ms can already be overlapped with it within a single device; hence an extra GPU provides no benefit. This rules out GPU contention as the limiting factor and confirms that the fixed per-step decode cost is the true bottleneck (Section 6.3).

6.6 The Role of CUDA Graph Replay

A subtle but decisive engineering factor is *CUDA graph replay*. We measure the target decoder's steady-state step time with and without `torch.compile` (max-autotune). The eager (uncompiled) decoder costs 21.9 ms per step, while the compiled decoder with CUDA graph replay costs 9.0 ms per step—a 2.4× reduction (Table 7). The reason is that `torch.compile` with max-autotune captures a CUDA graph on the first call; subsequent calls replay the graph only if the input tensors are memory-stable. Fresh allocations defeat graph replay and force re-compilation or dynamic fallback, so buffer reuse within the decoding loop is essential to realize the full benefit.

This explains a large part of the 0.42 → 2.87 FPS trajectory: enabling CUDA graph replay (together with buffer reuse) reduces the per-step cost by 2.4×, and

eliminating the implicit GPU–CPU synchronizations further reduces stalls. However, even after these optimizations, the ∼9 ms/step kernel cost remains and is the fundamental limit of the diagonal-decoding loop.

7 Discussion

Our study clarifies the conditions under which frame-level speculation pays off for world models. The central finding is that each diagonal decoding step carries a fixed ∼9 ms GPU kernel cost independent of how many tokens it decodes, so the ∼60 steps per frame alone account for the observed throughput. The draft (46 ms) can be overlapped with this cost, and a dual-GPU control experiment confirms that moving the draft to a second GPU does not help. Two directions remain promising: (i) *token-level draft*—feed the draft directly with VQ tokens, eliminating the VAE decode and depth pre-processing that dominate its latency; and (ii) *reducing the fixed per-step decode cost*—e.g., fusing the small-batch attention kernels or batching multiple diagonal steps, since the current per-step kernel time is nearly independent of the number of tokens decoded, indicating substantial fixed overhead that dominates the marginal compute.

8 Conclusion

We presented a frame-level speculative decoding framework for real-time interactive world models, together with a systematic empirical study of its speed–quality trade-offs. We showed that careful engineering can bring speculation to speed parity with the diagonal-decoding baseline (1.012× mean speedup over 197 paired clips), at a measurable PSNR cost, and we precisely characterized the two architectural obstacles that bound further gains. Our GPU-time breakdown reveals a fixed ∼9 ms per-step decode cost as the fundamental limit, and our dual-GPU control and tolerance mechanism provide a foundation for future acceleration of autoregressive world models—pointing to kernel-level optimization of the small-batch diagonal-decoding step, rather than draft or host-side overhead, as the next frontier.

Acknowledgments

We thank the MineWorld authors for releasing their code and checkpoints.

References

[1] MineWorld: A Real-Time and Open-Source Interactive World Model for Minecraft. *arXiv preprint arXiv:2504.08388*, 2025.

[2] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *ICML*, 2023.

[3] C. Chen, S. Borgeaud, G. Irving, et al. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

[4] L. Yang, B. Kang, Z. Huang, et al. Depth Anything: Unleashing the power of large-scale unlabeled data. In *CVPR*, 2024.